GALTOSM Handbook

Shounak Saha, Susmoy Das and Arpit Sharma

Table of Contents

Contents

1 Workflow of GALTOSM **1**

1.1 CADP to STORM/PRISM 1

1.2 mCRL2 to STORM/PRISM 1

1.3 PRISM/STORM to CADP 1

2 Grammar of Logics **2**

2.0.1 APCTL [5] [4] 2

2.0.2 APCTL* [5] 3

2.0.3 APRCTL [5] 4

2.0.4 PCTL [5] 5

2.0.5 PCTL* [5] 5

2.0.6 PRCTL [5] 6

3 Case Studies **6**

3.1 CADP Case Studies 7

3.1.1 **Ant on a Grid:** 7

3.1.2 **Lost Boarding Pass Problem:** 7

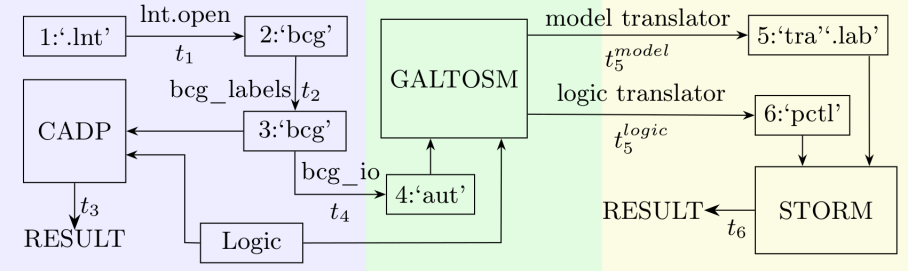
3.1.3 **Bounded Retransmission Protocol (BRP):** 8

3.1.4 **Gambling Problem:** 9

1 Workflow of GALTOSM

1.1 CADP to STORM/PRISM

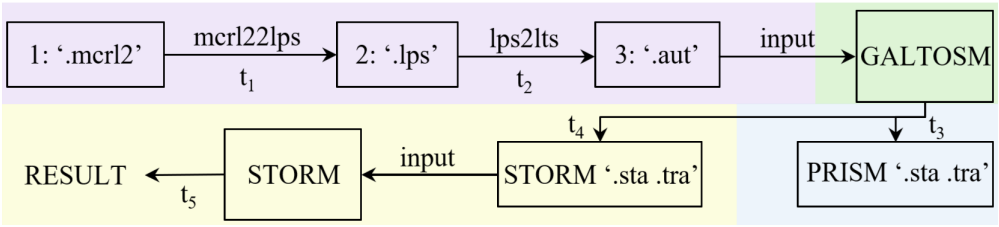
The workflow is explained in details in the main paper.



1.2 mCRL2 to STORM/PRISM

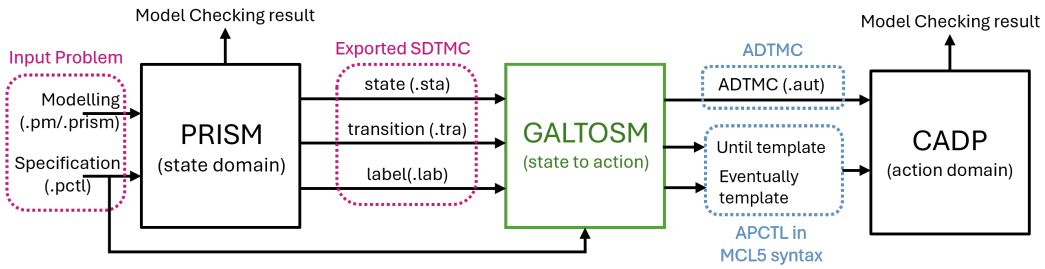
To perform state-based model checking of an mCRL2 specification, the following workflow is applied. The process begins by specifying the problem in the mCRL2 specification language and saving it as a .mcrl2 file. This file is then compiled to construct the model using the *mcrl22lps* command, which produces a *.lps* file; the time required for this step is denoted as $_1$. Subsequently, the *lps2lts* command converts the generated *.lps* file into a *.aut* file, with the corresponding execution time recorded as t_2. The combined time of $t_1 + t_2$ is the model building time in mCRL2 tool. It is important to note that the syntax of the resulting *.aut* file differs from that of the *.aut* format generated by CADP.

The .aut file is then provided as input to the GALTOSM tool, which produces machine readable formats of state domain tools, namely *.tra* and *.lab* files, depending on the selected target tool during execution. The time required to generate PRISM compatible files is denoted by $_3$, while the time to generate STORM compatible files is denoted by t_4. These generated files can subsequently be imported into their respective tools to perform model checking against PCTL formulas. The model checking time for STORM is shown in the figure and it's is recorded as t_5. A similar approach is followed when performing model checking using the PRISM tool.



1.3 PRISM/STORM to CADP

PRISM/STORM takes as input the probabilistic behavior specified in a *.prism/.pm* file for model construction. PRISM can generate a model and export it in the form of Machine Readable Files, consisting of three files, *.sta*, which stores state information (values of the state variables); *.tra*, which stores the probabilities of transitions between states; and *.lab*, which stores the labels of states. Alternatively, a PRISM model can be constructed by manually creating these three files. Next these three files are given as input to our tool GALTOSM to generate the .aut file compatible with the CADP toolset. Along with the ADTMC model, it also exports some nescessary data required for MCL formula generation.



2 Grammar of Logics

This section provides a detailed explanation on how to write syntactically correct formulas using the logics discussed in the paper, emphasizing on proper bracketing and the modified syntax used for AP(R)CTL and APCTL*. Since these logics share many structural elements, we only explain the APCTL syntax in detail. For the other logics, we only highlight the differences and necessary modifications required for each of them. A set of grammar rules is also provided to help the user generate valid formulas.
Throughout the section, state formulas are represented by ϕ, and ψ has been used to represent either the run or path formula based on the context of the underlying grammar. Run(s) and path(s) have been used interchangeably, representing the same formula. χ represents a valid auxiliary logic formula as defined in [4].

2.0.1 APCTL [5] [4]

✔ Grammar:

ϕ ::= `(true)` | `(false)` | (!ϕ) | (ϕ&ϕ) | (ϕ | ϕ) | ($P\lambda x[\psi]$)

ψ ::= `X_`(χ)ϕ | `X_tau`ϕ | ϕ`_`(χ)`U`ϕ | ϕ`_`(χ)`U_`(χ)ϕ

λ ::= `=?` | `>=` | `>` | `<=` | `<`

χ ::= `true` | `false` | `!`χ | `"a"` | (χ&χ) | (χ | χ)

✔ Note:

- `x`∈[0,1] if λ is not `=?`.
- `a` is an action name, which has to be enclosed within "double quotes"

✔ Details:

1. All state formulas(ϕ) are enclosed within `()`.
   Example `(true)`, `(P=?[path formula])`
2. (!ϕ) is the unary boolean NOT operator followed by the operand which is represented in literature by the symbol ¬. The syntax of the formula is `(!(state formula))`.
   Note: As the whole formula is a state formula, there will be `()` around the whole formula, and after `!` there is a state formula so there will also be `()` too.
   Example `(!(true))`, `(!(P=?[path formula]))`

3. The $(\phi\&\phi)$ operator is represented by the symbol &, and $(\phi \mid \phi)$ by |. The syntax of the formula is `((state formula)&(state formula))` and `((state formula)|(state formula))` respectively.
   Note: As the whole formula is a state formula, there will be `()` around the whole formula, and on either side of `&`/| there are state formulas, so there will also be `()`.
   Example `((true)&(true))`, `((true)|(P=?[path formula]))`
4. The $(P\lambda x[\psi])$ operator is represented by the symbol `P=?[path formula]`, `P>=0.36[path formula]`, or `P>.6[path formula]`, with similar forms for the less-than comparisons.
   Note: As the whole formula is a path formula, there will be `()` around the whole formula. The formula whose probability we want to calculate will be enclosed by `[]`. *Example* `(P=?[X_("action")(true)])`.
5. Run/path formulas are **not** enclosed in `()`.
   Example `X_tau(state formula)`, `(state formula)_(chi)U(state formula)`
6. X_(χ) operator is represented by `X_(chi)(state formula)`.
   Note: There will be `no ()` around the whole formula(rule 5). As this operator works on a state formula, the state formula will be enclosed by `()`. And χ will be enclosed within (),
   Example `(P>=[X_(“act1” | “act2”)(true)])`
7. `X_tau` operator is represented by `X_tau(state formula)`.
   Note: There will be `no ()` around the whole formula(rule 5). As this operator works on a state formula, the state formula will be enclosed by `()`.
   Example `(P>=[X_tau(true)])`
8. ϕ_(χ)Uϕ and ϕ_(χ)U_$(\chi)\phi$ operator is represented by `U` and the format is `(state formula)_(chi)U(state formula)` and `U_(chi)` with format `(state formula)_(chi)U_(chi)(state formula)`.
   Note: On either side of the operator, there are state formulas, so they will be enclosed in `()` and the chi's are enclosed in `()` *Example* `(P=?[(true)_("ACTION")U(true)])`, `(P=?[(true)_("ACTION1")U_("ACTION2")`

2.0.2 APCTL* [5]

✔ Grammar:

ϕ ::= `(true)` | `(false)` | $(!\phi)$ | $(\phi\&\phi)$ | $(\phi \mid \phi)$ | $(P\lambda x[\psi])$

ψ ::= ϕ | $(!\psi)$ | $(\psi\&\psi)$ | $(\psi \mid \psi)$ | $(\mathtt{X}\psi)$ | $(\mathtt{X_}(\chi)\psi)$ | $(\psi\mathtt{U}\psi)$

λ ::= `=?` | `>=` | `>` | `<=` | `<`

χ ::= `true` | `false` | `!`χ | `"a"` | $(\chi\&\chi)$ | $(\chi \mid \chi)$

✔ Note:

- `x`∈[0,1] if λ is not `=?`.
- `a` is an action name, which has to be enclosed within “double quotes”

✔ Details:
The rules for the state formulas are common since state formula operators are the same as those in APCTL, all operators has been described below

1. $(!\phi), (\phi\&\phi), (\phi \mid \phi), (P\lambda x[\psi])$ has been described in 1,2,3,4 respectively in section 2.0.1.

2. The run/path formulas (ψ) are enclosed in `()` too, contrary to rule 5 of APCTL grammar.
   Note: Rules for the state formula remains the same as 1, which will be enclosed by `()`.
   Example `(P=?[(X_tau(true))])`, `(P=?[((true)&(true))])`

3. ($!\psi$) operator has the same rules as state not formula. The operator representation is `(!(path formula)`.
   Note: The `()` is for the path/run formula where as the `()` is for the state formula.
   Example `(!(true))`

4. ($\psi\&\psi$) and($\psi \mid \psi$) has the same explanation as rule 3 of 2.0.1
   Note: The `()` is for the path/run formula where as the `()` is for the state formula. *Example* `(P=?[((!(true))&(true))])`

5. `X`(ψ) operator explanation is same as `X_tau` and can be found in rule 7 of 2.0.1
   The `()` is for the path/run formula whereas the `()` is for the state formula.
   Example `(P=?[(X(true))])`

6. (`X_`(χ)ψ) operator explanation can be found in rule 6 of 2.0.1
   The `()` is for the path/run formula whereas the `()` is for the state formula.
   Example `(P=?[(X_tau(true))])`

7. (ψ`U`ψ) operator is represented by `U` and the format is `(state formula)U(state formula)`.
   Note: On either side of the operator, there are state formulas, so they will be enclosed in `()`.
   Example `(P=?[(true)U(true)])`.

2.0.3 APRCTL [5]

✔ Grammar:

ϕ ::= `(true)` | `(false)` | ($!\phi$) | ($\phi\&\phi$) | ($\phi \mid \phi$) | ($P\lambda x[\psi]$) | ($E\lambda y[\phi]$)

ψ ::= `X_`(χ)ϕ | `X_tau`ϕ | ϕ`_`(χ)`U`ϕ | ϕ`_`(χ)`U_`(χ)ϕ

λ ::= `=?` | `>=` | `>` | `<=` | `<`

χ ::= `true` | `false` | `!`χ | `"a"` | ($\chi\&\chi$) | ($\chi \mid \chi$)

✔ Note:

- `x`$\in$[0,1] if λ is not `=?`.
- `y` $\in \mathbb{N}$
- `a` is an action name, which has to be enclosed within "double quotes"

✔ Details:
All state and run/path formulas coincide with the APCTL rule, which can be found in 2.0.1 except for the ($E\lambda y[\phi]$), which is explained below.

1. The ($R\lambda y[\phi]$) operator is represented by `E=?[]` or `E>=24[state formula]` or `E>64[state formula]` symbols and same for less than.
   Note: Since E operator has a state formula, `()` as per rule 1 of 2.0.1 inside the [] of E.
   Example `(E=?[(true)])`, `(E=?[(P>=0.5[X_tau(true)])])`

2.0.4 PCTL [5]

✔ Grammar:

ϕ ::= `(true)` | `(false)` | `(ap)` | (!ϕ) | (ϕ&ϕ) | (ϕ | ϕ) | ($P\lambda x[\psi]$)

ψ ::= `X`ϕ | ϕ`U`ϕ

λ ::= `=?` | `>=` | `>` | `<=` | `<`

✔ Note:

- `x`∈[0,1] if λ is not `=?`.
- `ap` is an atomic proposition, which has to be enclosed within "double quotes"

✔ Details:
The rules are the same as specified in 2.0.1 point 1,2,3,4,5.

1. All state formulas(ϕ) are enclosed within `()`.
   Example `(true)`, `(P=?[path formula])`
2. `(ap)` is an atomic proposition and is a state formula, so put it in `()` *Example* `("DEAD")`
3. The !ϕ operator is same as rule 2 of 2.0.1
4. The (ϕ&ϕ) operator and (ϕ | ϕ) operator is same as rule 3 of 2.0.1
5. The ($P\lambda x[\psi]$) operator is same as rule 4 of 2.0.1.
   Note: As the whole formula is a path formula, there will be `()` around the whole formula. The formula whose probability we want to calculate will be enclosed by `[]`.
   Example `(P=?[X(true)])`.
6. Run/path formulas are **not** enclosed in `()`.
   Example `X_tau(state formula)`, `(state formula)_(chi)U(state formula)`
7. `X`ϕ operator is represented by `X(state formula)`.
   Note: There will be `no ()` around the whole formula(rule 5). And as this operator works on a state formula, the state formula will be enclosed by `()`.
   Example `(P>=[X(true)])`
8. ϕ`U`ϕ is represented by `(state formula)U(state formula)`.
   Note: There will be no `(` around the whole formula but the state formulas in either sides of the `U` operator will be enclosed by `()`
   Example `(P=?[(true)U(true)])`

2.0.5 PCTL* [5]

✔ Grammar:

ϕ ::= `(true)` | `(false)` | `(ap)` | (!ϕ) | (ϕ&ϕ) | (ϕ | ϕ) | ($P\lambda x[\psi]$)

ψ ::= ϕ | (!ψ) | (ψ&ψ) | (ψ | ψ) | (`X`ψ) | (ψ`U`ψ)

λ ::= `=?` | `>=` | `>` | `<=` | `<`

✔ Note:

- `x`∈[0,1] if λ is not `=?`.
- `ap` is an atomic proposition, which has to be enclosed within "double quotes"

✔ Details:
Since the operators of PCTL* is a subset of APCTL*, the rules are the same as 2.0.2

2.0.6 PRCTL [5]

✔ Grammar:

ϕ ::= `(true)` | `(false)` | `(ap)` | (!ϕ) | (ϕ&ϕ) | (ϕ | ϕ) | ($P\lambda x[\psi]$) | ($R\lambda y[\phi]$)

ψ ::= `X`ϕ | ϕ`U`ϕ | $\phi \leq rU\phi$

λ ::= `=?` | `>=` | `>` | `<=` | `<`

✔ Note:

- `x`∈[0,1] if λ is not `=?`.
- `r,y` $\in \mathbb{N}$
- `ap` is an atomic proposition, which has to be enclosed within "double quotes"

✔ Details:
All state operators are common to the PCTL state operators in section 2.0.4 except the ($R\lambda y[\phi]$). The path/run formula of PRCTL has one additional operator from PCTL, which is $\phi \leq rU\phi$, they are explained below:

1. The ($R\lambda y[\phi]$) operator is represented by `R=?[]` or `R>=24[state formula]` or `R>64[state formula]` symbols and same for less than.
   Note: Since R operator has a state formula, `()` as per rule 1 of 2.0.1 inside the [] of E.
   Example `(R=?[(P>=1[X(true)])])`
2. $\phi \leq rU\phi$ operator is represented in the format `(state formula)<=rU(state formula)`. The rules of the until operator as defined in rule 8 in section 2.0.4
   Example `(R=?[(P>=0.1[("act1")<=25U("act2")])])`

3 Case Studies

To assess the efficiency and scalability of our approach, we conducted extensive experiments with several case studies involving randomised systems. All experiments were performed on a workstation equipped with an Intel Xeon(R) Silver 4314 CPU @ 2.40GHz × 64 processor, 640 GB of system memory, 1 TB SSD, and running Ubuntu 22.04.5 LTS. The default settings of the tools were used to evaluate our models. All reported times are in seconds, averaged from running the experiment 3 times. The experiments were carried out using the CADP 2026-b "Urbino" version, Storm 1.11.1 version and 202507.0 version of mCRL2. For all CADP-to-STORM case studies discussed in this section, we have followed the procedure mentioned in Section **??**. For the mCRL2-to-STORM experiments, we followed a procedure that is largely analogous to the one used for the CADP-to-STORM experiments[??]. Finally, for all mCRL2-to-PRISM case studies, we have followed the procedure described in [4].

3.1 CADP Case Studies

3.1.1 Ant on a Grid:

Richard Chatwin's well-known Ant on a Grid puzzle was originally published in The New York Times [1]. This puzzle was also modelled using the mCRL2 specification language [2]. In this problem, an ant is placed on a two-dimensional grid of size 8×8, starting at the intersection of the fifth vertical line and the third horizontal line, i.e., $(5, 3)$. The ant can move to an adjacent intersection along the grid lines, and it is equally likely to move in any one of the four directions. If the ant manages to reach the horizontal boundary, it survives; if the ant reaches the vertical boundary, it dies. The goal is to compute the probability that the ant survives. In our CADP model, we represent the different movements of an ant with six actions: LEFT, RIGHT, UP, DOWN, LIVE, and DEAD, each corresponding to its intuitive meaning. All states that can be reached by performing the actions LIVE or DEAD will have a self-loop with probability 1. We formalise the probabilistic property using APCTL as follows:

$$(P =?[(true)_(true)U_(\text{“}LIVE\text{”})(true)])$$

The corresponding PCTL formula obtained by applying GALTOSM is as follows:

$$(P =?[((\text{“}bot\text{”}\&(true)) \mid (!(\text{“}bot\text{”})\&(true)))U((!(\text{“}bot\text{”})\&(\text{“}LIVE\text{”}))\&P >= 1[X(\text{“}bot\text{”}\&(true))])])$$

Table 1: Results for the Ant on a Grid puzzle

Grid size	Model size (states, transitions)		Time (in seconds)						Prob
			CADP		GALTOSM		STORM	GALTOSM	
$n \times n$	ADTMC	SDTMC	lnt2bcg	MC	bcg2aut	action2state	MC	pipeline	
8	$4e1$, $1e2$	$1e2$, $2e2$	1.92	0.96	0.22	0.02	0.03	2.19	0.5862
16	$2e2$, $7e2$	$9e2$, $1e3$	1.95	0.95	0.28	0.02	0.03	2.28	0.5387
32	$9e2$, $3e3$	$4e3$, $7e3$	1.93	0.98	0.28	0.03	0.04	2.28	0.5182
64	$3e3$, $1e4$	$1e4$, $3e4$	1.94	1.17	0.35	0.07	0.13	2.49	0.5088
128	$1e4$, $6e4$	$7e4$, $1e5$	2.01	2.87	0.24	0.11	0.69	3.05	0.5043
256	$6e4$, $2e5$	$3e5$, $5e5$	2.1	41.93	0.29	0.37	9.18	11.94	0.5021
512	$2e5$, $1e6$	$1e6$, $2e6$	2.65	724.88	0.58	1.58	142.87	147.68	0.5010
1024	$1e6$, $4e6$	$5e6$, $8e6$	5.24	8230	1.55	6.85	1528.17	1541.81	0.5005
2048	$4e6$, $1e7$	$2e7$, $3e7$	20	66079.33	5.29	26.15	19450.57	19502.01	0.5002

We varied the size of the grid and also moved the initial position of the ant by the same factor to analyse the query. The results are shown in Table 1. The entries in the column 'Prob' refer to the result of the query. The entries in the 'ADTMC' and 'SDTMC' columns provide the approximate number of states and transitions in the original model and the embedded model, respectively. The entries in the CADP, 'lnt2bcg' column refer to $(t_1 + t_2)$ time as shown earlier in Fig. **??**. The CADP 'MC' column notes the t_3, the CADP model checking time. The GALTOSM 'bcg2aut' column records the t_4 time. The model and logic conversion time are jointly captured by $t_5 = t_5^{model} + t_5^{logic}$ in the 'action2state' column. The column 'STORM MC' stores the model building and model checking time required by STORM, i.e. t_6. Finally, 'GALTOSM pipeline' column records the total time required for specifying the problem in CADP, applying model and logic embeddings, and model checking in STORM, i.e., $t_1 + t_2 + t_4 + t_5 + t_6$. The same convention has been followed for all the case studies discussed in the following subsections.

3.1.2 Lost Boarding Pass Problem:

There is a plane with N seats. The first passenger to board the plane has lost his/her boarding pass and selects a seat at random. Every subsequent passenger takes his/her assigned seat if its available; otherwise,

they randomly select a seat from the remaining unoccupied seats. The question is: What is the probability that the last passenger to board the plane ends up sitting in his/her originally assigned seat? The problem has been modelled to capture all the possibilities that might occur during the boarding process when k passengers resolve their seats among themselves, such that from the $(k+1)^{th}$ passenger, everyone will sit in their own seat for $k \in [1, N-1]$, where N denotes the total number of passengers. Otherwise, if the k-th passenger finds out his/her seat has already been taken, he/she can select any seat among the $(N-k)$ available seats with equal probability of $\frac{1}{N-k}$. We have formalised our query using APCTL as follows:

$$(P =?[(true)_(true)U_(\text{“}LAST_GOT_SEAT\text{”})(true)])$$

and on converting this to PCTL we get:

$$(P =?[((\text{“}bot\text{”}\&(true)) \mid (!(\text{“}bot\text{”})\&(true)))U((!(\text{“}bot\text{”})\&(\text{“}LAST_GOT_SEAT\text{”}))\&P >= 1[X(\text{“}bot\text{”}\&(true))])])$$

Table 2: Results for the Lost Boarding Pass Problem

N	Model size (states, transitions)		Time (in seconds)						Prob
			CADP		GALTOSM		STORM	GALTOSM	
	ADTMC	SDTMC	lnt2bcg	MC	bcg2aut	action2state	MC	pipeline	
10^2	1e2, 2e2	2e2, 3e2	2.11	0.95	0.28	0.02	0.11	2.52	0.5
10^3	1e3, 2e3	2e3, 3e3	2.13	0.97	0.28	0.02	0.03	2.46	0.5
10^4	1e4, 2e4	2e4, 2e4	2.30	1.04	0.38	0.05	0.05	2.78	0.5
10^5	1e5, 2e5	2e5, 2e5	4.22	2.05	0.87	0.35	0.22	5.66	0.5
10^6	1e6, 2e6	2e6, 2e6	14.1	12.45	6.38	3.57	1.96	26.01	0.5
10^7	1e7, 2e7	2e7, 2e7	70.22	198.29	12.47	38.05	20.15	140.89	0.5
10^8	1e8, 2e8	2e8, 2e8	1433.15	3167.13	64.55	436.25	207.95	2141.9	0.5

Table 2 shows the results of our query for different values of N. Note that the result of this query is $\frac{1}{2}$ irrespective of the value of N.

3.1.3 Bounded Retransmission Protocol (BRP):

We model a variant of the bounded retransmission protocol [3,6], where a sender transmits a file to a receiver in discrete frames. Each frame requires acknowledgment before the sender proceeds to the next. A frame is deemed successfully delivered only upon receipt of an acknowledgment. The sender's transmission channel achieves a 98% success rate, while the receiver's acknowledgment channel succeeds 99% of the time. If an acknowledgment is not received, the sender retries the frame up to 10 times. Should all attempts fail, the frame transmission is considered unsuccessful, halting further frame transmissions. Using both CADP and GALTOSM combined with STORM, we analysed the probability of successfully transmitting a file of n frames with a maximum of two retransmission attempts per frame. We have formalised the query using APCTL as follows:

$$(P =?[(true)_(\text{“}ATTEMPT\text{”} \mid \text{“}SUCCESS_FRAME\text{”} \mid \text{“}ACK_FAIL\text{”} \mid \text{“}RETRY1\text{”} \mid \text{“}RETRY2\text{”})U_(\text{“}SUCCESS_FILE\text{”})(true)])$$

and upon conversion the corresponding PCTL formula is:

$$(P =?[((\text{“}bot\text{”}\&(true)) \mid (!(\text{“}bot\text{”})\&(\text{“}ATTEMPT\text{”} \mid \text{“}SUCCESS_FRAME\text{”} \mid \text{“}ACK_FAIL\text{”} \mid \text{“}RETRY1\text{”} \mid \text{“}RETRY2\text{”})))U((!(\text{“}bot\text{”})\& (\text{“}SUCCESS_FILE\text{”}))\&P >= 1[X(\text{“}bot\text{”}\&(true))])])$$

The results, presented in Table 3, illustrate this probability for varying values of n.

Table 3: Results for the Bounded Retransmission Protocol

No. of frames	Model size (states, transitions)		Time (in seconds)						Prob
			CADP		GALTOSM		STORM	GALTOSM	
	ADTMC	SDTMC	lnt2bcg	MC	bcg2aut	action2state	MC	pipeline	
$1e1$	$2e2$, $4e2$	$5e2$, $7e2$	1.92	0.97	0.34	0.02	0.03	2.31	0.9997
$1e2$	$2e3$, $4e3$	$5e3$, $7e3$	1.95	0.97	0.22	0.03	0.03	2.23	0.9974
$1e3$	$2e4$, $4e4$	$5e4$, $7e4$	1.99	1	0.24	0.11	0.07	2.41	0.9739
$1e4$	$2e5$, $4e5$	$5e5$, $7e5$	2.46	1.30	0.48	0.87	0.44	4.25	0.7675
$1e5$	$2e6$, $4e6$	$5e6$, $7e6$	8.33	4.94	1.88	8.74	4.31	23.26	0.0709
$1e6$	$2e7$, $4e7$	$5e7$, $7e7$	230.32	122.42	15.11	95.44	43.3	384.17	3.2e-12
$1e7$	$2e8$, $4e8$	$5e8$, $7e8$	3332.71	2529.06	151.74	970.66	445.76	4900.87	1.2e-115
$5e7$	$1e9$, $2e9$	$2e9$, $3e9$	20513.40	14341.33	731.11	5153.81	2447.35	28845.67	2.8e-319

3.1.4 Gambling Problem:

This problem is inspired by the classic gambler's ruin problem in probability theory, where a gambler starts with an initial amount of money and aims to reach a target amount. In this variant, the gambler can participate in up to k simultaneous games, each with a probability p of winning and $1-p$ of losing. Winning a game increases the gambler's funds by a fixed amount, while losing deducts a fixed amount. The gambler may also choose to skip a game, incurring a penalty smaller than the loss from an unsuccessful play, and can opt to sit out multiple consecutive games. However, participation in a game requires sufficient funds to cover a potential loss. The process ends when the gambler's funds reach or exceed the target amount or fall below the amount needed to cover a single loss. We model a scenario where a player starts with an initial balance of \$10 and aims to reach a target by participating in multiple simultaneous games, choosing 3 or 5 parallel games. Each game has a 0.7 probability of winning and a 0.3 probability of losing, with a 0.999 probability of playing and a 0.001 probability of sitting out a game. A win yields a gain of \$8, a loss deducts \$6, and sitting out incurs a penalty of \$2. To validate the model, we compute the probability that the player depletes their funds or reaches the target, which should be 1, as these are the only terminal states. The APCTL formalisation for the same is as follows:

$$(P =?[(true)_(true)U_(\text{“}HOME\text{”} \mid \text{“}DRUNK\text{”})(true)])$$

and the corresponding PCTL formula is:

$$(P =?[((\text{“}bot\text{”}\&(true)) \mid (!(\text{“}bot\text{”})\&(true)))U((!(\text{“}bot\text{”})\&(\text{“}HOME\text{”} \mid \text{“}DRUNK\text{”}))\&P >= 1[X(\text{“}bot\text{”}\&(true))])])$$

Table 4: Results for the Gambling problem

Target amount	No. of games	Model size (states, transitions)		Time (in seconds)					
				CADP		GALTOSM		STORM	GALTOSM
		ADTMC	SDTMC	lnt2bcg	MC	bcg2aut	action2state	MC	pipeline
10^5	3	$1e5$, $3e5$	$2e5$, $4e5$	2.4	2.19	0.43	0.52	0.23	3.58
10^5	5	$1e5$, $4e5$	$2e5$, $5e5$	2.57	2.45	0.46	0.63	0.27	3.93
10^6	3	$1e6$, $3e6$	$2e6$, $4e6$	6.68	13.5	1.15	5.89	2.04	15.76
10^6	5	$1e6$, $4e6$	$2e6$, $5e6$	8.34	92.27	1.44	6.44	2.47	18.69
10^7	3	$1e7$, $3e7$	$2e7$, $4e7$	85.15	258.12	11.37	63.89	20.65	181.06
10^7	5	$1e7$, $4e7$	$2e7$, $5e7$	103.38	292.92	12.47	71.91	25.02	212.78
10^8	3	$1e8$, $3e8$	$2e8$, $4e8$	1702.73	4118.67	96.59	654.5	210.89	2664.71
10^8	5	$1e8$, $4e8$	$2e8$, $5e8$	1718.72	4519	125.4	759.56	254.18	2857.86

Table 4 compares the performance of CADP, and GALTOSM combined with STORM, analysing the time taken as the target amount and number of parallel games vary. Both model checkers returned a probability of 1, confirming the validity of the model. We analysed several key specifications to gain valuable insights. For instance, the probability of reaching a target amount increases as the number of games played rises. Specifically, if a gambler starts with 10 and aims to accumulate 10^6, the probability of success is 0.05 for one game, 0.38 for three games, and 0.56 for five games played concurrently. Additionally, we calculated the probability of a player choosing to sit out the first game (with only the initial balance) and still achieving the $\$10^6$ target by playing five games in parallel, which is 0.00055.

References

[1] Antonick, G.: Ant on a grid. Wordplay : The Crossword Blog of The New York Times (2013)

[2] Bunte, O., Groote, J.F., Keiren, J.J.A., Laveaux, M., Neele, T., de Vink, E.P., Wesselink, W., Wijs, A., Willemse, T.A.C.: The mcrl2 toolset for analysing concurrent systems. In: TACAS. vol. 11428, pp. 21–39. Springer International Publishing (2019)

[3] D'Argenio, P., Jeannet, B., Jensen, H., Larsen, K.: Reachability analysis of probabilistic systems by successive refinements. In: PAPM/PROBMIV'01. LNCS, vol. 2165, pp. 39–56. Springer (2001)

[4] Das, S., Sharma, A.: On the use of model and logical embeddings for model checking of probabilistic systems. In: FORTE. LNCS, vol. 13910, pp. 115–131. Springer (2023)

[5] Das, S., Sharma, A.: Embeddings between state and action based probabilistic logics. Form. Asp. Comput. **37**(2) (March 2025)

[6] Helmink, L., Sellink, M.P.A., Vaandrager, F.W.: Proof-checking a data link protocol. In: Types for Proofs and Programs. pp. 127–165. Springer, Berlin, Heidelberg (1994)